

Figure 3. Accuracy of state probe and state parity probe across layers on S_3 and S_5 models sometimes match signatures for AA, and sometimes PAA. In all models, the state probe accuracy increases roughly exponentially with model depth. We find that in PAA models, the parity of the state is linearly decodable from earlier intermediate layers, while in the AA models shown above, the parity is never linearly encoded in any layer of the model. (In other AA models, the parity can only be linearly decoded at the final layer.)

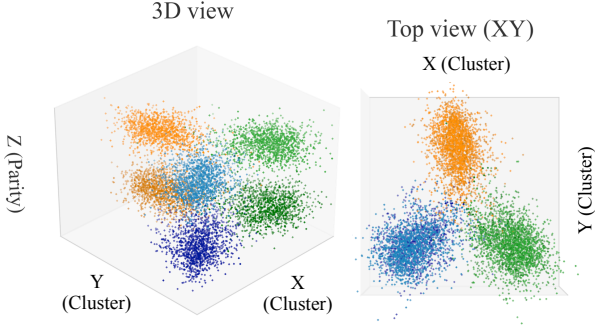


Figure 4. In models that learn PAA on S_3 , representations of the final product can be geometrically decomposed into two orthogonal directions, corresponding to the *parity* of the product (represented as the Z-axis in the above graph) and *cluster identity* of the product (represented by the X-Y plane). Note that the *clusters* are at 60 degrees to each other, and products of different parities within a cluster are equidistant from each other, with odd-parity products in one plane, and even-parity products in another plane.

colored cells in Figure 2) results specifically in predictions with incorrect parity. We find standard deviations to be low in Figure 9, confirming the robustness of our signatures.

4.3. Probing

Test set accuracies of linear probes across LM layers l are plotted in Figure 3. We report standard deviations of these accuracies in Table 1, which are all less than 10^{-3} . We again find empirical signatures consistent with those predicted by AA and PAA, on both S_3 and S_5 . Models with AA-type probing signatures always have AA-type patching signatures, and vice versa. Throughout the rest of this paper, we refer to models (and state-tracking mechanisms) as “AA-type” or “PAA-type” based on which cluster of signatures they exhibit. Results in Section C break down probe accuracies by sequence length, confirming that models solve sequences of exponentially longer length at deeper layers.

What exactly is the “non-parity residual” for PAA models? We visualize the linear components of representations near the final layer(s) of PAA models trained on S_3 . The rep-

resentations of states can be cleanly decomposed into two orthogonal parts: the *parity* of the product and a residual *cluster identity*, forming a triangular prism. In Figure 4, we project representations from the PAA model for each of the six states onto these components. Even-parity states (darker colors) and odd-parity states (lighter colors) are symmetric. The three cluster “spokes” are spaced 60 degrees apart.²

4.4. Generalization by Sequence Length

We next evaluate the state and parity accuracy of AA- and PAA-type models for held-out inputs of varying length. In general, we find that models learn to generalize perfectly to sequences of up to the length of their training data, then face a steep accuracy dropoff after (which we refer to as the “*cutoff length*”), rather than generalizing uniformly across all sequence lengths.

In Figure 5, where we plot the cutoff lengths at which each accuracy dips below 98%. We find that for models that learn PAA, the *parity accuracy cutoff length* is much longer than the *state accuracy cutoff length*, whereas, for models that learn an AA-type mechanism, the two cutoff lengths are equal. Furthermore, models that learn an AA-type mechanism tend to generalize better overall.

4.5. Attention Patterns

We look at attention patterns of LMs and check whether they can be used to differentiate between PAA models and AA models. Specifically, we find that in the early layers, PAA models exhibit *parity heads*, heads that place attention to *odd-parity* actions. Recall that the parity of a state can be determined by counting the number of odd-parity actions, and taking the parity of the count (Equation (1)). Examples of the parity head attention pattern are shown in Figure 15. We find no evidence of parity heads in any layer of AA models. (See Section F for a formal metric measuring how much an attention head behaves like a parity head.) We also

²Further details, including an analysis of S_5 , can be found in Section D.

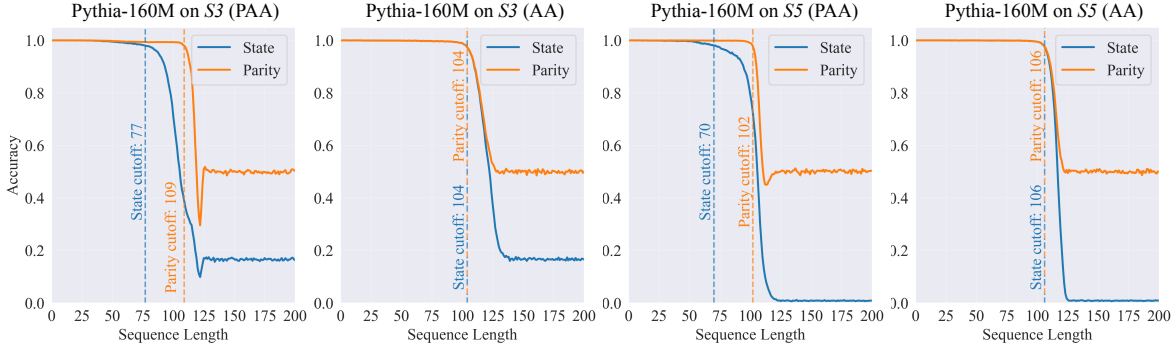


Figure 5. Generalization curves showing state and parity prediction accuracy as sequence lengths vary. Models are trained on length-100 sequences and asked to generalize to varying lengths of sequences. We plot generalization curves for AA and PAA models on S_3 and S_5 . In each plot, we show the 98% cutoff threshold, the sequence length at which accuracy dips below 98%. In the models that learned PAA, the parity cutoff is larger than the state cutoff, while in models that learned AA, the parity cutoff equals the state cutoff. Generally speaking, models that learned AA generalize better than ones that learned PAA.

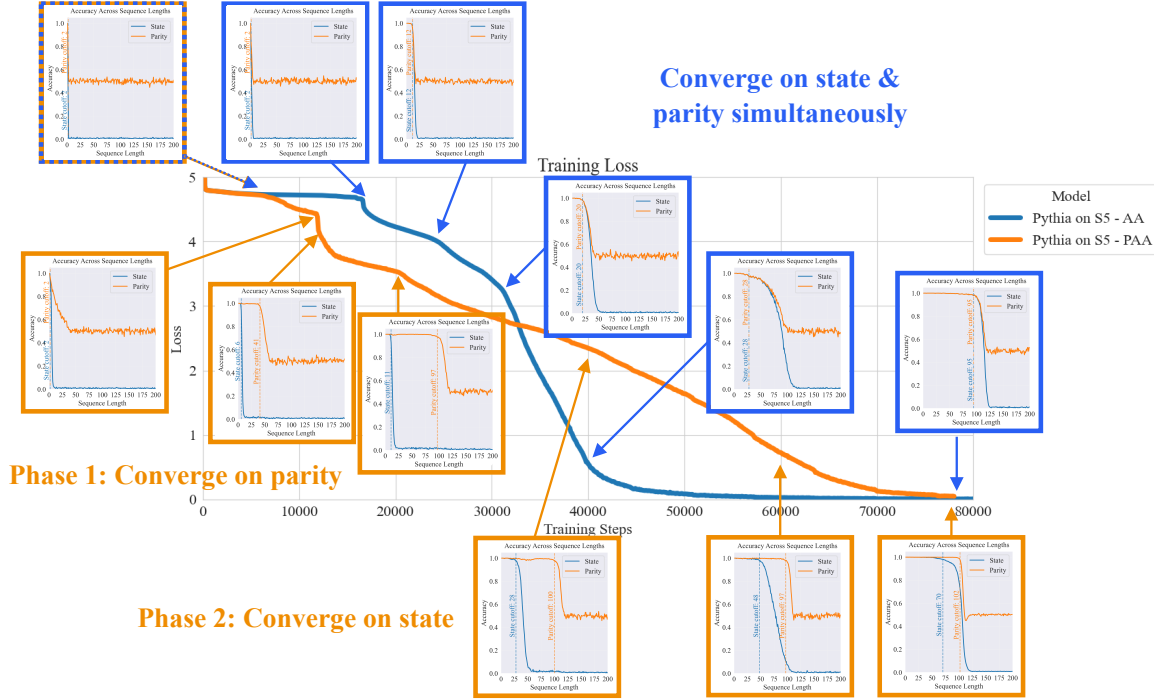


Figure 6. Annotated training curves for models that learn the AA and PAA algorithms. In PAA models (blue), we find that convergence happens in two phases: in the first phase, they learn to generalize parities up to sequence length 100, and in the second, they learn to generalize the states. In AA models (orange), parities and states are learned simultaneously. Note that AA models also tend to converge faster to (ultimately) a lower loss than PAA models.

find evidence that attention patterns in AA models sparsify in later layers of the network, forming a tree-like pattern expected of AA, shown in Figure 16.

5. Why do Transformers Learn One Mechanism or Another?

Having determined that trained models consistently exhibit AA- or PAA-like signatures, we next study the factors that

determine *which* mechanism emerges during training.

5.1. When in Training Do Distinct Mechanisms Arise?

We find that an LM’s eventual mechanism can be identified very early in training, based on the pattern of prediction errors. As in Figure 6, LMs that eventually learn AA improve the quality of their parity and state predictions in lockstep, while LMs that learn PAA learn in two phases: they first converge on learning *parity* over the entire length of the

training sequence; and only then do they learn to accurately predict the *state itself*.³

Because it is possible to identify these patterns early in training, our subsequent experiments classify LMs as AA-type or PAA-type based on generalization curves (Section 4.4) after 10k training steps, rather than waiting for the full probing and patching signatures to emerge.

5.2. What Factors Affect Which Mechanism is Learned?

Whether an LM learns AA or PAA is a deterministic function of four factors: model architecture, size, initialization scheme, and fine-tuning data order. Our next experiments evaluate each of these factors in turn. We explore two different model architecture families of various sizes (GPT-2, Radford et al., 2019, and Pythia, Biderman et al., 2023), several different model initializations (pre-trained on the Pile and trained from scratch with different random initializations), and up to 12 different data ordering seeds.

We find that model architecture and initialization, rather than model size, are the biggest determining factors of what mechanism the model chooses to learn. Figure 7 shows the ratio of LMs that learn each mechanism, aggregated by model architecture and initialization. The low variance indicates a minimal effect of model size.⁴ GPT-2 models, pre-trained or not, are split roughly evenly between the two mechanisms, while Pythia models tend to learn AA when pre-trained on the Pile, and PAA when not.

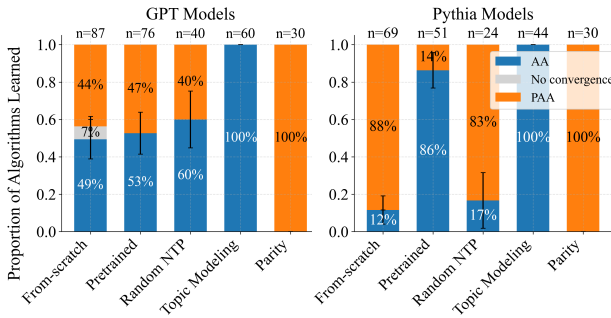


Figure 7. Proportion of GPT-2 and Pythia models that learn an AA-type mechanism, a PAA-type mechanism, or neither under different training regimes described in Section 5.

5.3. How Does Pre-training Affect Mechanism Choice?

We show that appropriately designed intermediate tasks can encourage models to learn one mechanism or the other.

Topic Modeling As a controlled way of studying how the next-token-prediction (NTP) objective affects which

³Experimental details can be found in Section G.

⁴A finer-grained breakdown of the ratio over each model size can be found in Figure 17.

mechanism LMs converge to, we generate length-100 documents with only S_3 elements as vocabulary items, and pre-train (randomly initialized) LMs with NTP on these documents, before training them on S_3 . Specifically, the documents are generated from a topic model with parameters: # of topics = 4, $\alpha = 0.3$, and $\beta = 0.1$, where α is the density of topics in each document and β is density of words in each topic. As shown in Figure 7, when from-scratch LMs are trained with our topic modeling NTP objective, they always learn an AA-type mechanism.

Parity Prediction We first train the entire model on predicting the **state parity** of the sequence to output token 1 if odd and 0 if even, before transitioning to training on the actual S_3 objective. In Figure 7, we show that we can induce GPT-2 and Pythia models to learn PAA when trained from scratch on parity. Notably, from-scratch Pythia models already tend to learn PAA as a baseline behavior. Therefore, we also apply this curriculum on Pythia models pre-trained on the Pile, and find that it consistently converts the mechanism learned from AA-type to PAA-type.⁵

Control: Random Next-Token-Prediction As a control, we train LMs on length-100 documents of *random* S_3 elements sampled from a uniform distribution. We confirm that the control fine-tuning did not affect the ratio with which LMs learned each mechanism.

6. Conclusion

We have shown that LMs trained on permutation tracking tasks learn one of two distinct mechanisms: one consistent with an “associative algorithm” (AA) that composes action subsequences in parallel across successive layers; and another with a “parity-associative algorithm” (PAA) which first computes a shallow parity heuristic in early layers and then computes a residual to the parity using an associative procedure. LMs that learn an AA-type mechanism tend to generalize better and converge faster; different choices of model architecture and training scheme encourage the discovery of one mechanism over another.

While a large number of other state tracking tasks can be reduced to the more complex permutation task we study (S_5), our experiments leave open the question of whether the specific mechanisms LMs use to solve S_5 are also deployed for these other tasks.

Impact Statement

The S_3 and S_5 tasks we choose to study in this paper can be generalized to many different state tracking scenarios funda-

⁵We discuss another parity curriculum using an extra parity loss term in Section H.3.